

MACHINE LEARNING IN FINANCE

MODULE 2 | LESSON 4

PRINCIPAL COMPONENT ANALYSIS AND INTEREST RATE MODELING

Reading Time	30 Minutes
Prior Knowledge	PCA, Machine learning
Keywords	Yield curves, covariance, VaR, Loadings

In the previous lesson, we studied principal component analysis (PCA) and the mathematical theory behind it. In this lesson, we will implement PCA on treasury rates and use the resultant principal components to calculate the Value at Risk.

1. Introduction

1.1 Yield Curve

A yield curve is a line plot of bond interest rates of the same credit quality, having different maturities. The yield curve has two dimensions to it:

- It depicts investors' average market expectations on the performance of future short-term bonds.
- The term premium - When the yield curve is upward sloping, it shows investors' confidence in the market as they are optimistic about receiving better returns in long-term bonds. In instances where the curve is downward sloping, investors expect tough times like a recession ahead. The yield curve could also be flat depicting no change in short-term and long-term rates.

These features of the yield curve can be modeled using principal components.

1.2 Value at Risk (VaR)

In this subsection, we only cover a small portion of VaR that will be useful to a later topic.

VaR is a useful metric to measure market risk on portfolio performance. VaR can be thought of as the largest possible loss we can incur for a given confidence interval and time period. VaR is a numerical value that distinguishes the tail (loss) of a distribution from the rest of the distribution.

An individual position-level VaR is calculated by multiplying the position volatility (standard deviation) of return, the portfolio total value, the absolute of proportion and the z -score of our chosen confidence interval.

z -score is used since we assume that our returns follow a normal distribution. For example, the z -score of a one-tailed 99% VaR can be read in the [normal table](#) as equal to 2.32635.

In this lesson we will focus on portfolio-level VaR, which is calculated as: $\text{Portfolio Var} = \text{Portfolio Volatility} \times \text{Portfolio Total Value} \times \text{z-score of confidence interval}$ where the portfolio volatility is the standard deviation of the returns in our portfolio.

2. Principal Component Analysis

Principal component analysis can be used to handle risk arising from data where variables are highly correlated. In this approach, we consider the historical data describing the market variables' movement and define factors that will explain the movements.

We will consider yield data with maturities of 1 month, 3 months, 6 months, 1 year, 2 years, 3 years, 5 years, 7 years, 10 years, 20 years, and 30 years.

2.1 Loading Packages

We start by loading helper packages that will help us achieve our tasks.

```
In [ ]: # Load Libraries
# Global Libraries
# Disable the warnings
import warnings
from datetime import datetime

import numpy as np
import pandas as pd
import pandas_datareader.data as web
import scipy as sp

# Plotting
import seaborn as sns
from matplotlib import pyplot
from sklearn.preprocessing import StandardScaler

warnings.filterwarnings("ignore")

from sklearn.decomposition import PCA
```

2.2 Loading Data

We download U.S. yield data ranging from 1-month to 30-year rates.

```
In [ ]: # downloading the data
start = datetime(2002, 6, 30)
end = datetime(2022, 6, 30)
data = [
    "DGS1M0",
    "DGS3M0",
    "DGS6M0",
    "DGS1",
    "DGS2",
    "DGS3",
    "DGS5",
    "DGS7",
    "DGS10",
    "DGS20",
    "DGS30",
]
data = web.DataReader(data, "fred", start, end).dropna(how="all").ffill()
```

```
In [ ]: data.rename(
    columns={
        "DGS1M0": "1m",
        "DGS3M0": "3m",
        "DGS6M0": "6m",
        "DGS1": "1y",
        "DGS2": "2y",
        "DGS3": "3y",
        "DGS5": "5y",
        "DGS7": "7y",
        "DGS10": "10y",
        "DGS20": "20y",
        "DGS30": "30y",
    },
    inplace=True,
)
df = data.copy()
df.head(10)
```

2.3 Exploratory Data Analysis

```
In [ ]: df.shape
```

```
In [ ]: df2 = df.copy()
```

Now let's visualize the movement of the yield curves.

```
In [ ]: df.plot(figsize=(15, 8))
pyplot.ylabel("Rate")
pyplot.legend(bbox_to_anchor=(1.01, 0.9), loc=2)
pyplot.suptitle(
    "Fig. 1: Yield Curve Movement", fontweight="bold", horizontalalignment="right"
)
pyplot.show()
```

3. Generating Principal Components

In this section, we study how to generate principal components from a given dataset.

3.1 Using the Covariance Matrix

The day-to-day data from financial markets usually have two properties

- Noise
- Signal

When we apply PCA to the dataset, we can extract its signal and find the minimum quantity of data that will account for the largest percentage of the whole dataset.

We then detrend the data by standardizing it into z-scores.

```
In [ ]: df_std = (df - df.mean()) / df.std()  
df_std.head()
```

Let us derive the covariance matrix of the standardized data above.

```
In [ ]: cov_matrix_array = np.array(np.cov(df_std, rowvar=False))  
cov_matrix_array
```

```
In [ ]: cov_df = pd.DataFrame(cov_matrix_array, columns=df.columns, index=df.columns)  
cov_df
```

Then, perform eigendecomposition on the covariance matrix and find the percentage of the eigenvectors as a percentage of the total variance.

```
In [ ]: # Perform eigendecomposition  
  
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix_array)  
  
# Put data into a DataFrame and save to excel  
df_eigval = pd.DataFrame({"Eigenvalues": eigenvalues})  
  
eigenvalues
```

```
In [ ]: # We calculate explained variance  
  
explained_variance = [round(variance / sum(eigenvalues), 3) for variance in eigenvalues]  
explained_variance
```

```
In [ ]: # Save output to Excel  
columns = [  
    "PC1",  
    "PC2",  
    "PC3",  
    "PC4",  
    "PC5",  
    "PC6",  
    "PC7",  
    "PC8",  
    "PC9",  
    "PC10",  
    "PC11",  
]
```

```

]
df_eigvec = pd.DataFrame(eigenvectors, columns=columns, index=df.columns)

df_eigvec.to_excel("df_eigvec.xlsx")
eigenvectors[0]

```

```
In [ ]: df_eigvec
```

The indices in the table above are the maturities of the rates that we considered for this exercise while the columns in our dataframe are the factor loadings describing the rate movements.

Working out the combined variation of the components:

```
In [ ]: from itertools import accumulate

df_eigval["Explained proportion"] = df_eigval["Eigenvalues"] / np.sum(
    df_eigval["Eigenvalues"]
)
df_eigval["Cumulative Explained Variance"] = list(
    accumulate(df_eigval["Explained proportion"])
)

# Format as percentage
df_eigval.style.format({"Explained proportion": "{:.2%}"})
df_eigval.style.format({"Cumulative Explained Variance": "{:.2%}"})

```

```
In [ ]: fig, ax = pyplot.subplots()

pyplot.plot(df_eigvec["PC1"])
pyplot.suptitle(
    "Fig. 2: The First Factor Loading", fontweight="bold", horizontalalignment="
)
pyplot.ylabel("Factor Loadings")
ax.set_xticks(np.arange(11))
ax.set_xticklabels(df.columns, rotation="vertical")
pyplot.xlabel("Term")
pyplot.show()

```

The first factor loading (PC1) is the weighted combination of all the rates and represents a parallel shift in our yield curve. We can see that for one unit of the loading, the 1-month rate increases by \$0.299\$ basis points, the 2-month rate increases by \$0.302\$ basis points, and so on.

```
In [ ]: fig, ax = pyplot.subplots()

pyplot.plot(df_eigvec["PC2"])
pyplot.suptitle(
    "Fig. 3: The Second Factor Loading", fontweight="bold", horizontalalignment="
)
pyplot.ylabel("Factor Loadings")
ax.set_xticks(np.arange(11))
ax.set_xticklabels(df.columns, rotation="vertical")
pyplot.xlabel("Term")
pyplot.show()

```

The second factor loading (PC2) represents a change of slope of the yield curve when the short end moves in the opposite direction of the long end zone. We can observe that the rates for 1 month to 4 years move in one direction and the rates between 5 years and 30 years move in the other direction.

```
In [ ]: fig, ax = pyplot.subplots()

pyplot.plot(df_eigvec["PC3"])
pyplot.suptitle(
    "Fig. 4: The Third Factor Loading", fontweight="bold", horizontalalignment="
)
pyplot.ylabel("Factor Loadings")

ax.set_xticklabels(df.columns, rotation="vertical")
pyplot.xlabel("Term")
pyplot.show()
```

The third factor loading (PC3) represents a twist of the sovereign yield curve, which happens when the short- and long-end segments move up simultaneously as the yield moves down.

We can now plot the scree plot to visualize the factor loadings' contribution to the variance of the dataset. /+

```
In [ ]: PC_values = np.arange(11)
pyplot.plot(
    PC_values, df_eigval["Explained proportion"] * 100, "o-", linewidth=2, color
)
pyplot.plot(
    PC_values,
    df_eigval["Cumulative Explained Variance"] * 100,
    "o-",
    linewidth=2,
    color="red",
)
pyplot.suptitle("Fig. 5: Scree Plot", fontweight="bold", horizontalalignment="ri
pyplot.title("Scree Plot")
pyplot.xlabel("Principal Component")
pyplot.ylabel("Variance Explained")
pyplot.legend(["Individual Variance", "Cumulative Variance"])
pyplot.show()
```

We can see that the first two components describe more than 99% of the variance and can therefore be used to describe the yield curve movement.

Now let's calculate the principal components, which are the dot product of the standardized data and the eigenvectors.

```
In [ ]: principal_components = df_std.dot(eigenvectors)
principal_components.columns = df_eigvec.columns
principal_components.head()
```

We now plot the first three components, which account for more than 99% of the explained variance in the data.

```
In [ ]: pyplot.plot(principal_components[["PC1", "PC2", "PC3"]])
pyplot.xlabel("Time (Years)")
pyplot.suptitle(
    "Fig. 6: Principal Component", fontweight="bold", horizontalalignment="right"
)
pyplot.legend(["PC1", "PC2", "PC3"], bbox_to_anchor=(1.01, 0.9), loc=2)
pyplot.show()
```

The principal components do not tell us much, but from the diagram above, we can see that the first principal component is more volatile than the other two, which is expected.

Below is a function that sums up the PCA process we have seen above.

```
In [ ]: def PCA_CALC(df, num_reconstruct):
    df -= df.mean(axis=0) / df.std()
    R = np.cov(df, rowvar=False)
    eigen_values, eigen_vectors = sp.linalg.eigh(R)
    eigen_vectors = eigen_vectors[:, np.argsort(eigen_values)[::-1]]
    eigen_values = eigen_values[np.argsort(eigen_values)[::-1]]
    eigen_vectors = eigen_vectors[:, :num_reconstruct]

    return np.dot(eigen_vectors.T, df.T).T, eigen_values, eigen_vectors
```

```
In [ ]: scores, evals, evecs = PCA_CALC(df, 11)
```

```
In [ ]: fig, ax = pyplot.subplots()

evecs = pd.DataFrame(evecs)
pyplot.plot(evecs.loc[:, 0:2])
pyplot.suptitle(
    "Fig. 7: The Factor Loadings", fontweight="bold", horizontalalignment="right"
)
pyplot.legend(["PC1", "PC2", "PC3"], loc="lower right")
pyplot.ylabel("Factor Loadings")
ax.set_xticks(np.arange(11))
ax.set_xticklabels(df.columns, rotation="vertical")
pyplot.xlabel("Term")
pyplot.show()
```

As discussed in the previous subsection, PC1 reflects a (mostly) parallel shift, PC2 reflects a tilt, and PC3 reflects a twist or butterfly.

We now work to reconstruct the original data from our factor loadings.

```
In [ ]: reconst = pd.DataFrame(np.dot(scores, evecs.T), index=df.index, columns=df.columns)

reconst.plot(figsize=(15, 8))
pyplot.ylabel("Rate")
pyplot.xlabel("Years")
pyplot.title("Reconstructed Mean-Subtracted Dataset")
pyplot.suptitle(
    "Fig. 8: Reconstructed Mean-SUBtracted Dataset",
    fontweight="bold",
```

```

        horizontalalignment="right",
    )
    pyplot.legend(bbox_to_anchor=(1.01, 0.9), loc=2)
    pyplot.show()

```

We now reconstruct our original dataset from the principal components and plot the outcome.

```

In [ ]: for cols in reconst.columns:
        reconst[cols] = reconst[cols] + df2.mean(axis=0)[cols]

reconst.plot(figsize=(15, 8))
pyplot.xlabel("Date")
pyplot.ylabel("Rates")
pyplot.title("Reconstructed Initial Dataset")
pyplot.suptitle(
    "Fig. 9: Reconstructed Initial Dataset",
    fontweight="bold",
    horizontalalignment="right",
)
pyplot.legend(bbox_to_anchor=(1.01, 0.9), loc=2)
pyplot.show()

```

Note that these steps can be implemented using a scikit learn module as shown below.

We start by computing the correlation between the term interests.

```

In [ ]: # correlation
correlation = df.corr()
pyplot.figure(figsize=(15, 15))
pyplot.title("Correlation Matrix")
pyplot.suptitle(
    "Fig. 10: Correlation Matrix", fontweight="bold", horizontalalignment="right"
)
sns.heatmap(correlation, vmax=1, square=True, annot=True, cmap="cubehelix");

```

Then, standardize the dataset as explained earlier. Note that when the variable scales are different, we use a correlation matrix, and when the variable scales are similar, we use a covariance matrix.

```

In [ ]: scaler = StandardScaler().fit(df)
rescaledddf = pd.DataFrame(scaler.fit_transform(df), columns=df.columns, index=df)
# summarize transformed data
df.dropna(how="any", inplace=True)
rescaledddf.dropna(how="any", inplace=True)
rescaledddf.head(2)

```

See the visualization of the scaled dataset below.

```

In [ ]: rescaledddf.plot(figsize=(14, 10))
pyplot.ylabel("Rate")
pyplot.legend(bbox_to_anchor=(1.01, 0.9), loc=2)
pyplot.suptitle(
    "Fig. 11: Scaled Yield Curve Plot", fontweight="bold", horizontalalignment="right"
)
pyplot.show()

```

We call the PCA algorithm and fit our data to it, then data transformation takes place here.

```
In [ ]: model = PCA()
PrincipalComponent = model.fit(rescaledddf)
```

We now visualize the explained variance of our components.

```
In [ ]: NumEigenvalues = 5
fig, axes = pyplot.subplots(ncols=2, figsize=(14, 4))
pd.Series(model.explained_variance_ratio_[:NumEigenvalues]).sort_values(
    ascending=False
).plot.bar(title="Explained Variance Ratio by Top Factors", ax=axes[0])
pd.Series(model.explained_variance_ratio_[:NumEigenvalues]).cumsum().plot(
    ylim=(0, 1), ax=axes[1], title="Cumulative Explained Variance"
)

# explained_variance
pd.Series(np.cumsum(model.explained_variance_ratio_)).to_frame(
    "Explained Variance_Top 5"
).head(NumEigenvalues).style.format("{:,.2%}".format)
pyplot.suptitle("Fig. 12: Scree Plots", fontweight="bold", horizontalalignment="right")
pyplot.show()
```

```
In [ ]: pyplot.plot(model.components_[0:3].T)
pyplot.xlabel("Principal Component")
pyplot.suptitle(
    "Fig. 13: Factor Loadings", fontweight="bold", horizontalalignment="right"
)
pyplot.legend(["PC1", "PC2", "PC3"], loc="lower right")
pyplot.show()
```

Again, let's not forget that PC1 reflects a (mostly) parallel shift, PC2 reflects a tilt, and PC3 reflects a twist or butterfly.

Let's create a dataframe having only the first 3 components, which is what we are interested in.

```
In [ ]: model = PCA().fit(rescaledddf)
columns = ["pca_comp_%i" % i for i in range(11)]
df_pca = pd.DataFrame(
    model.transform(rescaledddf), columns=columns, index=rescaledddf.index
)
df_pca.head()
```

```
In [ ]: # get the component variance
# Proportion of Variance (from PC1 to PC11)
model.explained_variance_ratio_
```

```
In [ ]: # Cumulative proportion of variance (from PC1 to PC6)
np.cumsum(model.explained_variance_ratio_)
```

```
In [ ]: # component loadings or weights (correlation coefficient between original variab
# component loadings represents the elements of the eigenvector
# the squared loadings within the PCs always sums to 1
```

```

loadings = model.components_
num_pc = model.n_features_
pc_list = ["PC" + str(i) for i in list(range(1, num_pc + 1))]
loadings_df = pd.DataFrame.from_dict(dict(zip(pc_list, loadings)))
loadings_df["variable"] = df.columns.values
loadings_df = loadings_df.set_index("variable")
loadings_df

```

```

In [ ]: # positive and negative values in component loadings reflects the positive and n
# correlation of the variables with the PCs. Except A and B, all other variables
# positive projection on first PC.

# get correlation matrix plot for loadings

import seaborn as sns

sns.set(rc={"figure.figsize": (11.7, 8.27)})
import matplotlib.pyplot as plt

ax = sns.heatmap(loadings_df, annot=True, cmap="Spectral")
plt.suptitle(
    "Fig. 14: Correlation matrix for Loadings",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()

```

```

In [ ]: # get eigenvalues (variance explained by each PC)
model.explained_variance_

```

So far, the three approaches we have applied have yielded similar results. So we want to plot the biplot of the first two principal components.

```

In [ ]: from pca import pca

```

```

In [ ]: # Or reduce the data towards 2 PCs
model = pca(n_components=2)

# Fit transform
results = model.fit_transform(rescaled_df)

```

```

In [ ]: # Make biplot with the number of features
print("\033[1m" + "Fig. 15: Biplot without the scores" + "\033[0m")
fig, ax = model.biplot(cmap=None, label=False, legend=False, figsize=(10, 6))

```

From the biplot, we can see short-term interest rates are highly correlated, and this can also be said of the long-term interest rates, this is because the angle between them is smaller. Comparing short-term and long-term interest rates, we can conclude that they are negatively correlated as the angle between them is wider. The mid-term interest rates seem not to be correlated with both the short- and long-term interest rates.

4. Computing Value At Risk (VaR) using PCA

In this section, we will demonstrate how we can use PCA to calculate VaR. For the purposes of this lesson, let us assume we have a portfolio with the exposures to interest rate variation as shown in the table below:

```
In [ ]: df_portfolio = pd.DataFrame(
    {
        "2 year rate": [10],
        "3 year rate": [4],
        "5 year rate": [-8],
        "7 year rate": [-7],
        "10 year rate": [2],
    }
)

df_portfolio
```

The table reads as follows: A \$1\$ basis point would see our portfolio value increase by \$10\$ million considering the 2-year rate; similarly, we will see a \$4\$ million portfolio increase in a 3-year rate, and so on.

From our scree plot, we saw that two-factor loadings were sufficient to model our yield rates as they account for more than 99% of the data variance. Recall our loadings dataframe.

```
In [ ]: df_eigvec
```

We can now calculate the exposure brought about by the first factor as

```
In [ ]: 10 * 0.3169 + 4 * 0.322 - 8 * 0.3231 - 7 * 0.3143 + 2 * 0.2987
```

The exposure due to the second factor will be

```
In [ ]: 10 * 0.1856 + 4 * 0.0994 + 8 * 0.0798 + 7 * 0.2122 - 2 * 0.3285
```

The first factor has very little exposure as compared to the second factor.

The change in the portfolio value becomes $\Delta P = 0.2695 f_1 + 3.7204 f_2$

Recall the loadings variance below:

```
In [ ]: df_eigval
```

We can calculate the standard deviation (factor score) as shown below:

```
In [ ]: df_eigval["Factor_store"] = np.sqrt(df_eigval["Eigenvalues"])
df_eigval
```

The standard deviation of ΔP will therefore be

```
In [ ]: np.sqrt(np.square(0.2695) * 9.304 + np.square(3.7204) * 1.553)
```

Finally, the one day 99% VaR is given by $\sigma(\Delta P) \times Z_{\alpha}$

5. Conclusion

In this lesson, we have applied principal component analysis concept on yield rate data and seen how we can arrive at the same conclusion using three different approaches. We have used results to calculate VaR for our portfolio.

References

1. Carcano, Nicola. "Yield Curve Risk Management: Adjusting Principal Component Analysis for Model Errors." *The Journal of Risk*, vol. 12, no. 1, 2009, pp. 3–16.
2. Oprea, Andrea. "The Use of Principal Component Analysis (PCA) in Building Yield Curve Scenarios and Identifying Relative-Value Trading Opportunities on the Romanian Government Bond Market." *Journal of Risk and Financial Management*, vol. 15, no. 6, 2022. <https://www.mdpi.com/1911-8074/15/6/247/htm>.
3. Stat Trek. "Normal Distribution Calculator." <https://stattrek.com/online-calculator/normal>.